

2º ESTÁGIO- POO: DATA. ____/____/____ PROFESSORA: Ayla Rebouças

ALUNO(A): _____

Considere um sistema para gerenciar vendas de salgados vendidos para clientes via WhatsApp. Cada cliente tem um número de telefone (ex: "(83)99991111") único e cada pedido tem um código único (ex: "P123"). Os salgados possuem tipo (como coxinha de frango) representado por um enum `TipoSalgado`. Um pedido pode incluir salgados de vários tipos. O sistema permite cadastrar novos pedidos de salgados, alterar o status do pedido (ex: indicando que foi entregue), alterar pedido, pesquisar os pedidos de um certo cliente, pesquisar os pedidos para um certo dia, etc. A seguir são apresentados trechos de algumas classes do sistema e um diagrama UML. Observação: No mapa (`Map`) utilizado para os pedidos, o código do pedido funciona como chave.



```

public enum TipoSalgado {
    COXINHA_DE_FRANGO, PASTEL_DE_CARNE, EMPADA_DE_FRANGO
}

import java.util.List;
public class Pedido {
    private String codigo;
    private String telefoneCliente;
    private List<ItemDePedido> itensDoPedido;
    private DataHora dataHoraEntrega;
    public Pedido(String codigo, String telefoneCliente, DataHora dataHoraEntrega,
List<ItemDePedido> itensDoPedido) {
        this.codigo = codigo;
        this.telefoneCliente = telefoneCliente;
        this.dataHoraEntrega = dataHoraEntrega;
        this.itensDoPedido = itensDoPedido;
    } // Demais métodos...
}

=====
public interface SistemaSalgados {
    void cadastrarPedido(String codigo, String telefoneCliente, DataHora dataHoraEntrega,
List<ItemDePedido> itensDoPedido) throws PedidoJaExisteException;
    Pedido pesquisarPedido(String codigo) throws PedidoNaoExisteException;
    List<Pedido> pesquisarPedidosDoCliente(String telefoneCliente);
    List<Pedido> pesquisarPedidosParaODia(int dia, int mes, int ano);
    void alterarPedido(String codigo, String telefoneCliente, DataHora dataHoraEntrega,
List<ItemDePedido> itensDoPedido) throws PedidoNaoExisteException;
    int pesquisarQuantidadeDeSalgadosDoTipoPedidosParaHorario(TipoSalgado tipo,
DataHora dataHora);
}

import java.util.HashMap;
import java.util.List;
import java.util.Map;
public class SistemaSalgadosMap implements SistemaSalgados{
    private Map<String, Pedido> pedidos;
    public SistemaSalgadosMap() {
        this.pedidos = new HashMap<String, Pedido>();
    }
    @Override
    public void cadastrarPedido(String codigo, String telefoneCliente, DataHora
dataHoraEntrega, List<ItemDePedido> itensDoPedido) throws PedidoJaExisteException {
        if (this.pedidos.containsKey(codigo)) {
            throw new PedidoJaExisteException("Já existe pedido com este código");
        } else {
            this.pedidos.put(codigo, new Pedido(codigo, telefoneCliente, dataHoraEntrega,
itensDoPedido));
        }
    }
    //...Demais métodos da classe considerando que implementa a interface e
    // que lança em seus métodos mesma exceções dos métodos da interface.
}

import static org.junit.jupiter.api.Assertions.*;
import java.util.ArrayList;
import org.junit.jupiter.api.Test;
public class SistemaSalgadosMapTest {
    @Test
    public void testaCadastrosEPesquisas() {
        SistemaSalgadosMap sistema = new SistemaSalgadosMap();
        try {
            List<ItemDePedido> itensDoPedido = new ArrayList<>();
            itensDoPedido.add(new ItemDePedido(TipoSalgado.COXINHA_DE_FRANGO, 20));
            itensDoPedido.add(new ItemDePedido(TipoSalgado.PASTEL_DE_CARNE, 50));
            sistema.cadastrarPedido("P111", "(83)99995555",
                new DataHora(11,8,2026,8,0), itensDoPedido);
            Pedido p = sistema.pesquisarPedido("P111");
            assertEquals("(83)99995555", p.getTelefoneCliente());
            // TODO: Acrescentar códigos aqui
        } catch (Exception e) {
            fail("Não deveria falhar");
        }
    }
}

```

1. (2.0) Implemente o método `pesquisarPedido` da classe `SistemaSalgadosMap` que retorna o `Pedido` cujo código é igual ao código passado como parâmetro para este método. Caso não seja encontrado nenhum pedido com o código, o método deve lançar a exceção `PedidoNaoExisteException`. Lembre-se de que o código de cada pedido funciona como chave do `Map` utilizado na classe `SistemaSalgadosMap`.
2. (2.0) Implemente o método `alterarPedido` da classe `SistemaSalgadosMap` que busca o pedido com o mesmo código indicado no parâmetro e atualiza todas as demais informações do pedido conforme os parâmetros passados para o método (Por exemplo, altera a `dataHoraEntrega`, o telefone do cliente e os itens, utilizando os métodos da classe `Pedido`). Caso não seja encontrado nenhum pedido com o código indicado, o método deve lançar a exceção `PedidoNaoExisteException`. Lembre-se de que o código de cada pedido funciona como chave do `Map` utilizado na classe `SistemaSalgadosMap`.
3. (2.0) Implemente o método `pesquisarPedidosParaODia` da classe `SistemaSalgadosMap`. Este método recebe como parâmetros 3 inteiros: o dia, o mês e o ano correspondentes à data a pesquisar. O método deve buscar nos pedidos cadastrados no sistema aqueles cuja `dataHoraEntrega` corresponde a esta data indicada pelos parâmetros. Dica: Veja os parâmetros na interface `SistemaSalgados`.
4. (2.0) Complemente o método de teste JUnit para a classe `SistemaSalgadosMapTest` para que:
 - a) Pesquise no sistema a quantidade de salgados do tipo COXINHA DE FRANGO pedidos para o dia 11/08/2026 às 8:00 e verifique com uma asserção se o valor da quantidade retornado foi 20 .
 - b) Pesquise no sistema a lista dos pedidos feitos para o cliente de telefone "(83)88889999" e verifique utilizando asserção se o tamanho da lista retornada é 0.

Obs: Coloque em sua resposta só o código que colocaria na parte do comentário de `//TODO` da classe `SistemaSalgadosMapTest`.

5.a) (2.0) Indique se cada afirmativa a seguir é verdadeira (V) ou falsa (F).

- () A classe `SistemaSalgadosMap` é um bom candidato a *Creator* de objetos do tipo `Pedido`.
- () É uma má prática de design OO deixar a responsabilidade de pesquisar pedidos na classe `SistemaSalgadosMap` já que esta classe não é o *expert* sobre pedidos.
- () Fazemos Refatoração quando a estrutura interna do código precisa de melhorias, apesar do sistema estar funcionando.
- () O uso de herança promove um acoplamento mais fraco entre as classes envolvidas se comparado ao uso de composição.
- () O refatoramento consiste em realizar modificações no código-fonte com o objetivo principal de alterar o seu comportamento externo e adicionar novas funcionalidades complexas.

5.b)(2.0) Implemente o método `pesquisarPedidosDoCliente` da classe `SistemaSalgadosMap` que retorna uma lista dos pedidos cujo número de telefone do cliente é igual ao número de telefone passado como parâmetro para este método.

Observações Gerais:

- A questão 5.b só precisa ser respondida se você quiser repor a 1a. unidade ou melhorar sua 1a. nota. Sugiro que faça de qualquer forma se estiver com tempo.
- Note que para obter a coleção de valores guardados no `HashMap` você pode utilizar o método `values()` e se quiser saber se um `Map` contém alguma chave com um certo valor, pode utilizar o método `containsKey(valorChave)`. O método `get(valorChave)` de `HashMap` retorna o objeto do mapa que é identificado pela chave passada ou `null` se o objeto não for encontrado. Colocamos um elemento em um `Map` com `put(chave, elemento)`
- Para saber quantos elementos há em uma `Collection` ou `List` você pode usar o método `size()`. `ArrayList` é um subtipo de `List`, que por sua vez é um subtipo de `Collection`. Adicionamos elementos em uma `Collection` com `add(elemento)`. Para fazer assertivas/asserções (verificações) em testes você pode usar:
`assertTrue(condicaoQueDeveSerVerdade);`
`assertEquals(valorEsperado, valorObtido);`
`assertFalse(condicaoQueDeveSerFalsa);`
- O `fail("Mensagem")` faz um teste falhar se atingir aquele ponto na execução.

- Para comparar objetos de um tipo enum você pode usar `==` ou `equals`
-